

Hierarchical Multi-Task Learning with Liquidity-Aware Signals for Stock Forecasting

Hengyi Yang^{1,*}, Sida Lin^{2,*}, Yiyang Qi^{3,*}, Yankai Chen⁴,
Haohan Zhang⁵, Xianhua Peng^{1,†}, Jian Guo^{3,†}

¹Peking University, Shenzhen, China ²The Chinese University of Hong Kong, Shenzhen, China

³International Digital Economy Academy (IDEA), Shenzhen, China ⁴Cornell University, Ithaca, United States

⁵The Hong Kong University of Science and Technology, Guangzhou, China

hengyiyanghy@gmail.com, sidalin1@link.cuhk.edu.cn, qiyiyang@idea.edu.cn, yankaichen@acm.org,

hzhang760@connect.hkust-gz.edu.cn, xianhuapeng@pku.edu.cn, guojian@idea.edu.cn

*Equal contribution †Corresponding authors ‡This work was conducted during his internship at IDEA Research.

Abstract—Stock price forecasting is a long-standing challenge in computational finance, driven by the inherent randomness of markets and complex temporal patterns. While recent deep-learning models have raised forecasting accuracy by jointly modeling inter-stock and temporal price dynamics, they conflate inter-stock relationships with intra-stock temporal dependencies and focus solely on the univariate objective of price movement. To address these limitations, we propose LiMT, a Hierarchical Multi-Task Learning framework that integrates liquidity-aware signals for stock price forecasting. LiMT employs a Market Regime Encoder (MRE) module that first extracts contemporaneous cross-stock dependencies, then models each stock’s temporal dynamics, yielding a unified latent state. Building on this latent state, we introduce a Liquidity-Driven Learning (LDL) module, a mixture-of-experts architecture that features cross-task gating mechanisms to jointly predict price movement, volatility, and trading volume. We further design an Adaptive Portfolio Optimization (APO) mechanism that converts multi-task forecasts into executable portfolio weights under transaction-cost and liquidity constraints. Extensive experiments on the CSI300 and CSI500 benchmarks show that LiMT performs best among strong neural and tree-based baselines across the reported metrics. In realistic CSI300 backtests, APO improves annualized return from 3.99% to 10.01% and Sharpe ratio from 1.22 to 1.86 over equal weighting, showing that the multi-task forecasts translate into deployable portfolio gains.

Index Terms—Regression, time series, financial data mining, machine learning

Code Availability: An anonymized repository is available at anonymous.4open.science/r/LiMT-F039

I. INTRODUCTION

Stock price forecasting is a central problem in quantitative finance because it directly affects portfolio construction and risk management. Unlike stationary time series, stock returns are highly non-stationary and jointly influenced by macroeconomic conditions, capital flows, investor sentiment, and unexpected shocks [1], [2]. These dependencies unfold along both temporal and cross-sectional dimensions, making effective forecasting inherently challenging [3].

Learning-based forecasting has advanced from decision trees and ensemble models [4]–[9] to recurrent, convolutional, and Transformer-style architectures that better capture temporal structure in financial signals [10]–[15]. To model inter-

stock structure, prior work introduced static relation graphs and later attention-based architectures that dynamically aggregate peer information [3], [11], [16]–[18]. Other extensions incorporate futures or sentiment signals [19], [20]. In parallel, market-microstructure studies show that volume and volatility are important for liquidity, execution, and risk control [21]–[24], yet these signals are still rarely learned jointly with return prediction in end-to-end stock forecasters.

Despite this progress, three structural limitations remain. First, many relation-aware architectures impose a fixed order between temporal aggregation and cross-sectional interaction [3], [11], [16]–[18]. If temporal compression happens first, day-specific peer effects are averaged away before cross-stock interaction; if cross-sectional mixing is followed by temporal aggregation, transient co-movements and stock-specific momentum become entangled. The resulting representation cannot cleanly separate contemporaneous market synchronization from within-stock dynamics. Second, most methods optimize return prediction alone, leaving volume and volatility outside the learning objective even though trading volume carries independent alpha content and volatility directly affects risk-adjusted performance and execution feasibility [24], [25]. Existing multi-task models [26], [27] remain largely return-centered and do not explicitly learn liquidity-sensitive auxiliary signals for downstream portfolio construction. Third, portfolio construction is often reduced to simple ranking or threshold rules that ignore liquidity limits, impact, and transaction costs, weakening the link between predictive accuracy and executable returns.

To address these issues, we propose LiMT, a Hierarchical Multi-Task Learning framework that integrates liquidity-aware signals for stock price forecasting. Our contributions are threefold.

Market Regime Encoder (MRE): We introduce a hierarchical stock transformer that applies cross-stock attention independently at each historical step and only then performs within-stock temporal modeling. This ordering preserves time-specific cross-sectional structure before temporal compression, reducing the signal entanglement induced by rigid sequential pipelines.

Liquidity-Driven Learning (LDL): We jointly learn return, volume shock, and volatility through a MoE-based multi-task module with cross-task gating, so informative liquidity and risk signals can be transferred to the return objective while conflicting auxiliary information is suppressed.

Adaptive Portfolio Optimization (APO): We design a lightweight liquidity-aware weighting rule that de-normalizes predicted volume and volatility, linearly fuses liquidity and risk preferences, and improves annualized return from 3.99% to 10.01% with Sharpe ratio rising from 1.22 to 1.86 in realistic backtests under participation and transaction-cost constraints.

II. PROBLEM FORMULATION

We study daily cross-sectional stock forecasting with three jointly learned targets aligned with both prediction and execution: next-day return, volume shock, and volatility. On trading day t , the model uses the historical feature window to predict the targets realized at $t + 2$.

A. Main Task - Return

The return metric, denoted as z_r , is computed as the relative change in closing prices over two consecutive time intervals, expressed as $z_r = \frac{\text{Close}(t+2)}{\text{Close}(t+1)} - 1$. This formula quantifies the percentage variation in closing prices, enabling the simultaneous capture of both the directional movement and magnitude of price changes.

B. Auxiliary Task 1 - Volume Shock

The volume shock metric, z_v , measures the deviation of trading volume from its recent historical trend. It is calculated as the difference between the volume at time step $t + 2$ and the five-period moving average of volume at time step $t + 1$, i.e., $z_v = \log(\text{volume}_{t+2}) - \frac{1}{5} \sum_{i=1}^5 \log(\text{volume}_{t+2-i})$. This metric is designed to identify abnormal trading activities, such as sudden surges in buying or selling pressure.

C. Auxiliary Task 2 - Volatility

The volatility metric is defined as $z_\sigma = \frac{\text{high}_{t+2} - \text{low}_{t+2}}{\text{vwap}_{t+2}}$, which measures price fluctuation intensity. This ratio normalizes the price range (difference between the high and low prices) by the volume-weighted average price (vwap), providing a standardized estimate of the magnitude of price swings within the given time frame.

All targets undergo cross-sectional Z -score normalization at each trading day: $ZS(y) = \frac{y - \mu_y}{std_y}$, where μ_y and std_y denote the cross-sectional mean and standard deviation of y , respectively.

III. METHODOLOGY

A. Overview

Figure 1 depicts the architecture of LiMT, which consists of three components. (1) The **Market Regime Encoder (MRE)** applies cross-stock attention at each historical step and then models within-stock temporal dynamics, so changing market conditions are captured implicitly through time-varying

interactions rather than explicit regime labels. (2) Building on MRE, the **Liquidity-Driven Learning (LDL)** module jointly predicts return, volume shock, and volatility through shared experts and gated cross-task transfer. (3) The **Adaptive Portfolio Optimization (APO)** module then converts these forecasts into time-varying long-short weights under liquidity and participation constraints.

B. Market Regime Encoder Module

To capture both cross-sectional and temporal information, the Market Regime Encoder module comprises two attention-based sub-modules. First, a cross-sectional attention layer enriches each time-step's representation by attending to contemporaneous information across assets. Second, a temporal attention encoder summarizes all information within the rolling window. This orthogonal decomposition enables more precise modeling of both market-wide synchronization and individual stock momentum patterns.

Feature Projection: Input features $\mathbf{X} \in \mathbb{R}^{S \times T \times F}$ are first projected to a d -dimensional hidden space to facilitate correlation modeling:

$$\mathbf{H}^* = \text{Linear}(\mathbf{X}) \in \mathbb{R}^{S \times T \times d}, \quad (1)$$

where S represents the number of stocks in each trading day, T denotes the rolling window size, and d is the hidden feature dimension.

1) **Cross-Stock Attention:** To simplify notation, we illustrate the computation for a single time step. For each time step k , we extract a cross-sectional slice $\mathbf{H}_c \in \mathbb{R}^{S \times d}$ from $\mathbf{H}^*$ to capture contemporaneous correlations across all stocks without accessing future temporal information. The multi-head attention mechanism assigns higher weights to stocks that are more relevant or similar at the given time step:

$$\mathbf{H}_c = \mathbf{H}^*[:, k, :] \in \mathbb{R}^{S \times d}, \quad (2)$$

For each attention head $i = 1, \dots, h$:

$$\mathbf{A}_c^{(i)} = \text{Softmax} \left(\frac{(\mathbf{H}_c \mathbf{W}^{Q,(i)})(\mathbf{H}_c \mathbf{W}^{K,(i)})^\top}{\sqrt{d_k}} \right) \in \mathbb{R}^{S \times S}, \quad (3)$$

$$\mathbf{H}_{cross}^{(i)} = \mathbf{A}_c^{(i)} \mathbf{H}_c \mathbf{W}^{V,(i)} \in \mathbb{R}^{S \times d_k}, \quad (4)$$

where $\mathbf{W}^{Q,(i)}, \mathbf{W}^{K,(i)}, \mathbf{W}^{V,(i)} \in \mathbb{R}^{d \times d_k}$ are projection matrices for head i with $d_k = d/h$ and $\mathbf{H}_{cross}^{(i)}$ is the hidden states of heads.

Multi-head concatenation and output projection given as:

$$\mathbf{H}_{cross}^k = \text{Concat}(\mathbf{H}_{cross}^{(1)}, \dots, \mathbf{H}_{cross}^{(h)}) \mathbf{W}^O \in \mathbb{R}^{S \times d}, \quad (5)$$

where $\mathbf{W}^O \in \mathbb{R}^{d \times d}$ is the output projection matrix and $\mathbf{H}_{cross}^k$ denotes the hidden state at the k -th time step within the look-back window, with $k \in \{1, 2, \dots, T\}$.

Residual Connection and Normalization: To stabilize training and preserve information flow, we apply residual connections with layer normalization:

$$\mathbf{H}_{cross}^k = \text{LayerNorm}(\mathbf{H}_c + \mathbf{H}_{cross}^k) \in \mathbb{R}^{S \times d}. \quad (6)$$

Hidden state with cross-stock information given as:

$$\mathbf{H}^* = \text{Concat}(\mathbf{H}_{cross}^k) \in \mathbb{R}^{S \times T \times d} \quad (7)$$

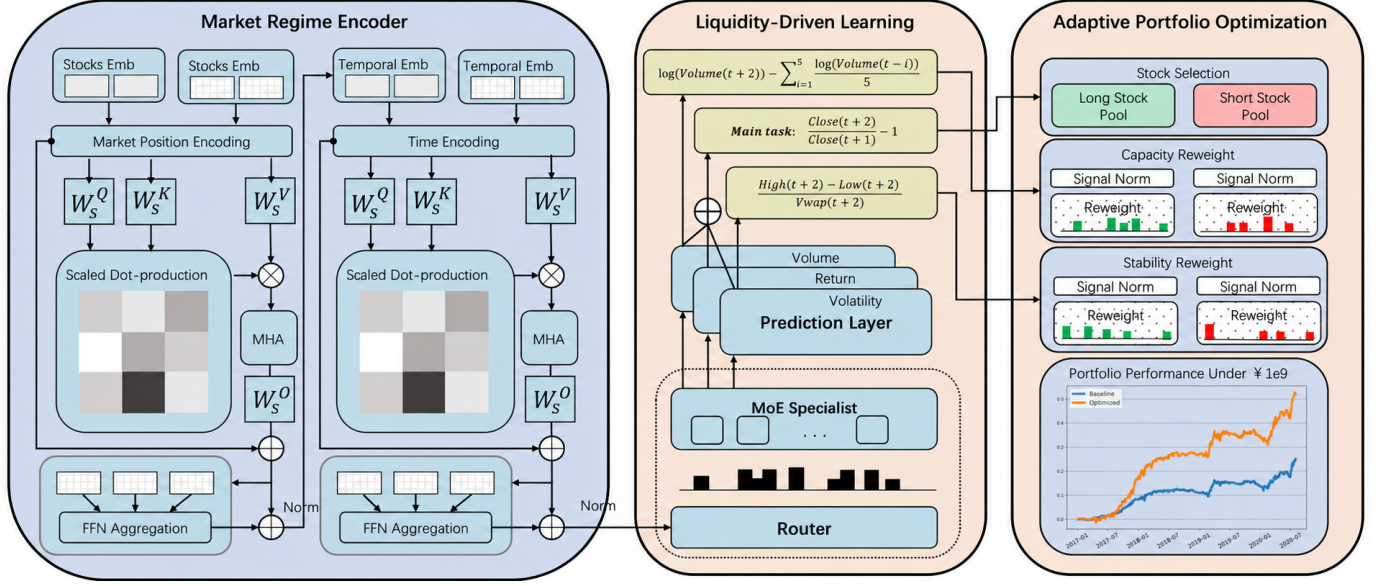


Fig. 1. Overview of the LiMT framework. Left: MRE module, Middle: LDL module, Right: APO module.

2) **Temporal-Stock Modeling**: After capturing cross-stock dependencies, we model temporal dynamics within each stock to extract time-series patterns and long-range dependencies. To simplify notation, we illustrate the computation for a single q -th stock. For each stock q , we extract a temporal slice $\mathbf{H}_t \in \mathbb{R}^{T \times d}$ from the cross-stock enhanced features to capture dependencies between past and recent time steps:

$$\mathbf{H}_t = \mathbf{H}^*[q, :, :] \in \mathbb{R}^{T \times d}, \quad (8)$$

For each attention head $i = 1, \dots, h$:

$$\mathbf{A}_t^{(i)} = \text{Softmax} \left(\frac{(\mathbf{H}_t \tilde{\mathbf{W}}^{Q,(i)})(\mathbf{H}_t \tilde{\mathbf{W}}^{K,(i)})^\top}{\sqrt{d_k}} \right) \in \mathbb{R}^{T \times T}, \quad (9)$$

$$\mathbf{H}_t^{(i)} = \mathbf{A}_t^{(i)} \mathbf{H}_t \tilde{\mathbf{W}}^{V,(i)} \in \mathbb{R}^{T \times d_k}, \quad (10)$$

where $\tilde{\mathbf{W}}^{Q,(i)}, \tilde{\mathbf{W}}^{K,(i)}, \tilde{\mathbf{W}}^{V,(i)} \in \mathbb{R}^{d \times d_k}$ are temporal projection matrices for head i and $\mathbf{H}_t^{(i)}$ is the output of different heads.

Multi-head concatenation and output projection given as:

$$\mathbf{H}_{temporal}^q = \text{Concat}(\mathbf{H}_t^{(1)}, \dots, \mathbf{H}_t^{(h)}) \tilde{\mathbf{W}}^O \in \mathbb{R}^{T \times d}, \quad (11)$$

where $\tilde{\mathbf{W}}^O \in \mathbb{R}^{d \times d}$ is the temporal output projection matrix and $\mathbf{H}_{temporal}^q$ denotes the hidden state of q -th stock of T s time step, with $q \in \{1, 2, \dots, S\}$.

Residual Connection and Normalization: Similar to cross-stock modeling, we apply residual connections:

$$\mathbf{H}_{temporal}^q = \text{LayerNorm}(\mathbf{H}_t + \mathbf{H}_{temporal}^q). \quad (12)$$

The representation for stock q uses the most recent temporal information:

$$\mathbf{h}^q = \mathbf{H}_{temporal}^q[-1, :] \in \mathbb{R}^d, \quad (13)$$

For the representation of all the stocks at T -th time given as:

$$\mathbf{h}^* = \text{Concat}(\mathbf{h}^q) \in \mathbb{R}^{S \times d}. \quad (14)$$

where $\mathbf{h}_s^q$ denotes the hidden state of q -th stock at the last time step, with $q \in \{1, 2, \dots, S\}$ and $\mathbf{h}^*$ is the concatenation of all stocks.

C. Liquidity-Driven Learning (LDL) Module

Capital flows are primarily reflected in changes in liquidity, including price, trading volume, and volatility. These three indicators capture complementary aspects of market dynamics and often exhibit regime-dependent behaviors. To jointly model these signals under varying market conditions, we introduce the Liquidity-Driven Learning (LDL) module. It integrates a Multi-gate Mixture-of-Experts architecture with a cross-task knowledge transfer mechanism to enhance return forecasting with liquidity-aware supervision.

1) **Multi-gate Mixture-of-Experts (MMoE)**: The MMoE module consists of expert routing and task-specific expert aggregation across multiple prediction objectives.

Expert Routing: Each task $k \in \{r, v, \sigma\}$, corresponding to return forecasting, volume anomaly prediction, and volatility estimation, is governed by an independent gating function:

$$\alpha^k = \text{Softmax}(\Phi_k(\mathbf{h}^*)) \in \mathbb{R}^{S \times n}, \quad (15)$$

where Φ_k denotes a task-conditioned routing function that learns the most relevant combinations of experts for each objective, and n is the number of experts.

Expert Aggregation: The task-specific representation is computed as a weighted combination of expert outputs:

$$\mathbf{o}_k = \sum_{e=1}^n \alpha^k[e] \cdot \text{Expert}_e(\mathbf{h}^*) \in \mathbb{R}^{S \times d}. \quad (16)$$

Feature Transformation: Before cross-task transfer, each task representation is projected into a shared latent space through dimensionality reduction and non-linear activation:

$$G_k(o_k) = \text{ReLU}(\text{Linear}_k(o_k)) \in \mathbb{R}^{S \times \frac{d}{2}}, \quad (17)$$

where $k \in \{r, v, \sigma\}$ and $\text{Linear}_k : \mathbb{R}^{S \times d} \rightarrow \mathbb{R}^{S \times \frac{d}{2}}$ denotes a task-specific projection layer.

2) **Cross-Task Knowledge Transfer:** To leverage auxiliary liquidity signals for improving return prediction, we introduce a stock-wise gating mechanism that adaptively transfers information from volume and volatility tasks to the primary return objective. This design reflects the financial intuition that volume shocks and volatility patterns may provide leading indicators of future price movements, while their usefulness varies across assets and market regimes.

The transfer strength from each auxiliary task is controlled by an independent scalar gate:

$$\begin{aligned} \beta_v &= \text{Sigmoid}(\text{Linear}_{v \rightarrow r}(G_v(o_v))) \in \mathbb{R}^{S \times 1}, \\ \beta_\sigma &= \text{Sigmoid}(\text{Linear}_{\sigma \rightarrow r}(G_\sigma(o_\sigma))) \in \mathbb{R}^{S \times 1}, \end{aligned} \quad (18)$$

where $\text{Linear}_{v \rightarrow r}(\cdot)$ and $\text{Linear}_{\sigma \rightarrow r}(\cdot)$ are two distinct projection functions. The coefficient β_v determines when volume-related liquidity signals should be emphasized for return forecasting, whereas β_σ controls the contribution of volatility-driven risk information. By separating these two gates, the model can capture heterogeneous cross-task relationships and selectively inject auxiliary knowledge when it is predictive, thereby reducing negative transfer.

Enhanced Representation: The return representation is enhanced through gated fusion of auxiliary task features:

$$o'_r = G_r(o_r) + \beta_v \odot G_v(o_v) + \beta_\sigma \odot G_\sigma(o_\sigma) \in \mathbb{R}^{S \times \frac{d}{2}}, \quad (19)$$

where $\odot$ denotes stock-wise broadcasting along the feature dimension. This formulation enables adaptive incorporation of complementary liquidity information into the return forecast-ing process.

Prediction: Each task employs a dedicated output layer to generate the final forecasts:

$$\begin{aligned} \hat{z}_r &= \text{Linear}_r(o'_r) \in \mathbb{R}^{S \times 1}, \\ \hat{z}_v &= \text{Linear}_v(G_v(o_v)) \in \mathbb{R}^{S \times 1}, \\ \hat{z}_\sigma &= \text{Linear}_\sigma(G_\sigma(o_\sigma)) \in \mathbb{R}^{S \times 1}. \end{aligned} \quad (20)$$

where $\hat{z}_r$, $\hat{z}_v$, and $\hat{z}_\sigma$ denote the predicted return, volume anomaly, and volatility values, respectively. The return prediction is computed from the enhanced representation o'_r with cross-task knowledge transfer, while auxiliary tasks are predicted from their own transformed representations.

3) **Multi-task Loss:** We optimize a weighted multi-task loss function using Mean Squared Error (MSE) on z-score normalized targets:

$$\mathcal{L} = \sum_{i \in \{r, v, \sigma\}} \lambda_i \cdot \text{MSE}(\hat{z}_i, z_i). \quad (21)$$

where $\lambda_r, \lambda_v, \lambda_\sigma$ are task-specific weights that balance the contribution of each loss item during training.

D. Adaptive Portfolio Optimization Module

Building on LDL, APO converts multi-task forecasts into executable long-short weights. At trading day t , stocks are sorted by $\hat{z}_r$, and the top and bottom deciles define the long and short candidate sets. For a candidate set $\mathcal{G}_t$, the auxiliary outputs are first de-normalized to physical units:

$$\begin{aligned} \hat{A}_{i,t+2} &= \exp(\hat{z}_{v,i,t} \cdot s_t^V + \mu_t^V + \bar{\ell}_{i,t}) \cdot p_{i,t}^{\text{vwap}}, \\ \hat{\sigma}_{i,t+2} &= \hat{z}_{\sigma,i,t} \cdot s_t^\sigma + \mu_t^\sigma, \end{aligned} \quad (22)$$

where μ_t^V, s_t^V and μ_t^σ, s_t^σ are the day- t cross-sectional moments of volume shock and volatility, $\bar{\ell}_{i,t} = \frac{1}{5} \sum_{k=0}^4 \log V_{i,t-k}$ is the 5-day mean of log-volume, and $p_{i,t}^{\text{vwap}}$ is the day- t VWAP.

We then apply the linear normalization

$$D_{\mathcal{G}_t}(x) = \sum_{k \in \mathcal{G}_t} (x_k - \min_{j \in \mathcal{G}_t} x_j), \quad (23)$$

$$\text{Norm}_{\mathcal{G}_t}(x_i) = \begin{cases} \frac{x_i - \min_{j \in \mathcal{G}_t} x_j}{D_{\mathcal{G}_t}(x)}, & \text{if } D_{\mathcal{G}_t}(x) > 0, \\ \frac{1}{|\mathcal{G}_t|}, & \text{otherwise,} \end{cases} \quad (24)$$

to form liquidity and risk weights:

$$\begin{aligned} w_i^A &= \text{Norm}_{\mathcal{G}_t}(\hat{A}_{i,t+2}), \\ w_i^\sigma &= \text{Norm}_{\mathcal{G}_t}(-\hat{\sigma}_{i,t+2}). \end{aligned} \quad (25)$$

The final executable weight is

$$w_i^* = \min \left(r \cdot w_i^\sigma + (1-r) \cdot w_i^A, \frac{\alpha \cdot A_{i,t}}{C} \right), \quad (26)$$

where $r \in [0, 1]$ balances risk control and liquidity preference, $\alpha = 2\%$ is the participation-rate cap, $A_{i,t}$ is realized turnover, and C is the portfolio capital.

IV. EXPERIMENTS

In this section, we conduct experiments to answer the following three research questions:

- **RQ1** How does LiMT compare with strong neural and tree-based baselines on real-world stock forecasting benchmarks?
- **RQ2** Which components of the proposed architecture drive the predictive gains?
- **RQ3** Do the auxiliary tasks and APO improve executable portfolio performance under realistic trading constraints?

1) **Datasets:** We evaluate LiMT on the Chinese A-share market using CSI300 and CSI500 constituents. CSI300 covers the largest and most liquid stocks, while CSI500 extends evaluation to a broader mid-cap universe. The data span 2008–2020, with training/validation/test splits of 2008-01-01 to 2014-12-31, 2015-01-01 to 2016-12-31, and 2017-01-01 to 2020-08-01. We use the public Alpha158 feature set from Qlib [28], with lookback window $\tau = 21$ and prediction horizon $w = 1$ day.

TABLE I
PERFORMANCE COMPARISON ON CSI300 AND CSI500 DATA. BOLD = BEST, UNDERLINE = SECOND-BEST.

Model	CSI300				CSI500			
	IC $\uparrow$	ICIR $\uparrow$	AR $_{\text{excess}}\uparrow$	IR $_{\text{excess}}\uparrow$	IC $\uparrow$	ICIR $\uparrow$	AR $_{\text{excess}}\uparrow$	IR $_{\text{excess}}\uparrow$
LSTM	0.0321 $\pm$ 0.01	0.2267 $\pm$ 0.04	0.0381 $\pm$ 0.03	0.5561 $\pm$ 0.46	0.0381 $\pm$ 0.00	0.3636 $\pm$ 0.02	0.0596 $\pm$ 0.04	1.0243 $\pm$ 0.64
GRU	0.0324 $\pm$ 0.00	0.2448 $\pm$ 0.05	0.0344 $\pm$ 0.02	0.5160 $\pm$ 0.25	0.0413 $\pm$ 0.00	<u>0.3962</u> $\pm$ 0.03	0.0723 $\pm$ 0.01	1.1982 $\pm$ 0.12
Transformer	0.0243 $\pm$ 0.00	0.1879 $\pm$ 0.01	0.0286 $\pm$ 0.00	0.4317 $\pm$ 0.10	0.0258 $\pm$ 0.00	0.2296 $\pm$ 0.03	0.0392 $\pm$ 0.02	0.6317 $\pm$ 0.26
ALSTM	0.0369 $\pm$ 0.00	0.2756 $\pm$ 0.03	0.0645 $\pm$ 0.04	0.9128 $\pm$ 0.55	0.0406 $\pm$ 0.00	0.3741 $\pm$ 0.04	0.0473 $\pm$ 0.02	0.7839 $\pm$ 0.26
SFM	0.0370 $\pm$ 0.00	0.2879 $\pm$ 0.04	0.0465 $\pm$ 0.02	0.5672 $\pm$ 0.29	0.0353 $\pm$ 0.00	0.3007 $\pm$ 0.04	0.0310 $\pm$ 0.00	0.2728 $\pm$ 0.03
TCN	0.0279 $\pm$ 0.00	0.2181 $\pm$ 0.01	0.0262 $\pm$ 0.02	0.4133 $\pm$ 0.25	0.0050 $\pm$ 0.00	0.0587 $\pm$ 0.00	-0.0682 $\pm$ 0.02	-1.0386 $\pm$ 0.26
TabNet	0.0283 $\pm$ 0.01	0.2168 $\pm$ 0.06	0.0278 $\pm$ 0.03	0.3767 $\pm$ 0.46	0.0341 $\pm$ 0.01	0.3187 $\pm$ 0.08	-0.0016 $\pm$ 0.06	0.0477 $\pm$ 0.97
TFT	0.0358 $\pm$ 0.00	0.2169 $\pm$ 0.03	0.0847 $\pm$ 0.02	0.8131 $\pm$ 0.19	0.0398 $\pm$ 0.01	0.2900 $\pm$ 0.04	0.0117 $\pm$ 0.01	0.0894 $\pm$ 0.09
Localformer	0.0321 $\pm$ 0.01	0.2469 $\pm$ 0.05	0.0225 $\pm$ 0.03	0.3310 $\pm$ 0.50	0.0313 $\pm$ 0.00	0.2827 $\pm$ 0.02	0.0250 $\pm$ 0.02	0.4120 $\pm$ 0.32
MASTER	0.0496 $\pm$ 0.01	0.4321 $\pm$ 0.00	0.1035 $\pm$ 0.01	1.2104 $\pm$ 0.01	<u>0.0435</u> $\pm$ 0.00	0.3862 $\pm$ 0.03	<u>0.1242</u> $\pm$ 0.00	1.3415 $\pm$ 0.00
XGBoost	0.0538 $\pm$ 0.00	0.3979 $\pm$ 0.00	0.0820 $\pm$ 0.00	0.9170 $\pm$ 0.00	0.0410 $\pm$ 0.00	0.3390 $\pm$ 0.00	0.0603 $\pm$ 0.00	0.7475 $\pm$ 0.00
CatBoost	0.0494 $\pm$ 0.00	0.3467 $\pm$ 0.01	0.0473 $\pm$ 0.00	0.3507 $\pm$ 0.01	0.0423 $\pm$ 0.00	0.3349 $\pm$ 0.00	0.0604 $\pm$ 0.00	0.7603 $\pm$ 0.00
LightGBM	0.0455 $\pm$ 0.00	0.3674 $\pm$ 0.00	0.1043 $\pm$ 0.00	1.2268 $\pm$ 0.00	0.0348 $\pm$ 0.00	0.3387 $\pm$ 0.00	0.1186 $\pm$ 0.00	<u>1.5247</u> $\pm$ 0.00
DoubleEnsemble	0.0521 $\pm$ 0.00	<u>0.4223</u> $\pm$ 0.01	<u>0.1158</u> $\pm$ 0.01	<u>1.3432</u> $\pm$ 0.11	0.0408 $\pm$ 0.00	0.3710 $\pm$ 0.01	0.0382 $\pm$ 0.00	0.1723 $\pm$ 0.00
LiMT	0.0575 $\pm$ 0.00	0.4462 $\pm$ 0.03	0.1809 $\pm$ 0.00	2.0414 $\pm$ 0.03	0.0496 $\pm$ 0.00	0.452 $\pm$ 0.02	0.1529 $\pm$ 0.00	2.0166 $\pm$ 0.03
Relative Improvement (%)	6.88%	5.41%	56.22%	51.98%	13.94%	14.08%	23.11%	32.26%

TABLE II
ABLATION STUDY ON CSI300 DATA.

Model	IC $\uparrow$	ICIR $\uparrow$	AR $_{\text{excess}}\uparrow$	IR $_{\text{excess}}\uparrow$
w/o Auxiliary Tasks	0.0543 $\pm$ 0.00	0.4035 $\pm$ 0.00	0.1734 $\pm$ 0.00	1.9418 $\pm$ 0.01
w/o LDL	0.0568 $\pm$ 0.00	0.4342 $\pm$ 0.01	0.1845 $\pm$ 0.01	1.9993 $\pm$ 0.00
w/o Auxiliary Tasks & LDL	0.0528 $\pm$ 0.00	0.3841 $\pm$ 0.02	0.1687 $\pm$ 0.00	1.8104 $\pm$ 0.00
w/o Cross-Stock Modeling	0.0493 $\pm$ 0.00	0.4130 $\pm$ 0.00	0.1277 $\pm$ 0.01	1.6929 $\pm$ 0.01
w/o Auxiliary Tasks & LDL & Cross-Stock Modeling	0.0489 $\pm$ 0.00	0.4060 $\pm$ 0.00	0.1049 $\pm$ 0.03	1.3838 $\pm$ 0.02

2) **Baselines:** We compare LiMT against fourteen representative baselines. The deep-learning group includes LSTM [29], GRU [30], Transformer [15], ALSTM [31], SFM [32], TCN [33], TabNet [34], TFT [35], Localformer [36], and MASTER [18]. The tree-based group includes XGBoost [7], CatBoost [37], LightGBM [8], and DoubleEnsemble [9].

3) **Evaluation:** We evaluate both predictive quality and trading value using IC, ICIR, excess annualized return, and excess information ratio; precise metric definitions are summarized in the appendix. The portfolio simulation follows a daily top-50 long strategy with 20 replacements per day.

4) **Implementation:** We implement LiMT using PyTorch on the Qlib platform [28]. Key hyperparameters are $d = 256$, $lr = 10^{-5}$, $head_{\text{cross}} = 4$, and $head_{\text{temp}} = 2$. Models are trained for up to 100 epochs with early stopping. Each experiment is repeated three times and reported as mean $\pm$ standard deviation. Complete hyperparameter tuning details, baseline configurations, and backtest setup are provided in Appendix A.

A. Overall Performance (RQ1)

Table I presents the comprehensive performance comparison across all baseline methods on both CSI300 and CSI500 datasets.

LiMT ranks first on all four metrics across both datasets. The gains are especially pronounced on the portfolio-oriented metrics: on CSI300, excess annualized return and excess information ratio improve by 56.22% and 51.98% over the strongest baseline, while on CSI500 the corresponding gains are 23.11% and 32.26%. Tree-based models, especially XGBoost, remain competitive on IC and ICIR, but LiMT translates predictions into stronger excess returns, suggesting that jointly modeling cross-sectional structure and liquidity-aware auxiliary signals is particularly useful for deployable decisions. Performance is also slightly more stable on CSI300 than on CSI500, which is consistent with the higher liquidity and lower heterogeneity of large-cap constituents.

Interpretability Analysis of Attention and Routing Mechanisms:

To illustrate how the framework uses cross-sectional, temporal, and task-specific structure, we conduct a case study on March 20, 2020 for the target stock SH600000. We visualize the cross-stock attention patterns produced by MRE, the cross-time propagation scores induced by temporal and cross-sectional attention, and the task-specific routing distributions over the shared LDL expert pool. For readability, we average attention weights across heads to obtain a cross-stock matrix $A_{c,j}$ at each historical step j and a temporal matrix $A_{t,u}$ for each stock u .

Figure 2 shows that the cross-stock attention of SH600000 over 100 randomly sampled peers is sparse and selective: only a small subset of stocks receives consistently high weights, and the attended peers vary across historical steps. This pattern is consistent with the view that relevant cross-sectional dependencies are time-varying rather than static.

Figure 3 further examines how temporal importance interacts with cross-sectional relations through cross-time propagation scores. Concretely, for a target stock u and a source stock v , we define the propagation strength at historical step j as

$$M[j, v] = \mathbf{A}_{t,u}[-1, j] \cdot \mathbf{A}_{c,j}[u, v], \quad (27)$$

where $\mathbf{A}_{t,u}[-1, j]$ denotes the temporal attention weight assigned by stock u when forecasting the last step of the rolling window, and $\mathbf{A}_{c,j}[u, v]$ represents the cross-stock attention weight from stock v to stock u at the j -th historical step. The first row of Figure 3 visualizes $M[j, v]$ over the lookback window and shows that peer influence is concentrated on a few historical steps rather than being spread uniformly across time.

The second row of Figure 3 reports cross-time propagation maps between SH600000 and SH600004, alternately treating each stock as source and target. The patterns are asymmetric and not strictly diagonal, suggesting that cross-asset interactions can be directional and temporally shifted rather than perfectly synchronized.

Finally, Figure 4 visualizes the gating weights of three representative stocks (SH600000, SH600005, and SH600008) across the Return, Volume Shock, and Volatility tasks. Because experts are shared across objectives, the distinct routing patterns across tasks illustrate how LDL allocates expert capacity differently for return, liquidity, and risk signals.

B. Ablation Study (RQ2)

To validate the effectiveness of each proposed component, we conduct systematic ablation studies by progressively removing key modules. Table II summarizes the results on CSI300 data.

- **w/o Auxiliary Tasks:** This variant removes the volume and volatility objectives and trains only on return prediction. Performance drops across all metrics, by 1.39 % to 7.80 %, showing that the auxiliary targets provide useful supervision beyond simple regularization.
- **w/o LDL:** This variant removes the LDL transfer module and feeds the MRE output directly to the prediction heads. The resulting gap to LiMT shows that simply adding auxiliary heads is not sufficient; gated cross-task transfer is a key part of the improvement.
- **w/o Auxiliary Tasks & LDL:** This variant removes both the auxiliary tasks and LDL. The broader degradation confirms that joint supervision and cross-task transfer are complementary.
- **w/o Cross-Stock Modeling:** This variant disables the cross-stock attention layer and therefore cannot use contemporaneous peer information. It causes the largest IC



Fig. 2. Cross-stock attention heatmap. The x-axis shows 100 sampled stocks, and the y-axis shows the 21-day lookback window on March 20, 2020.

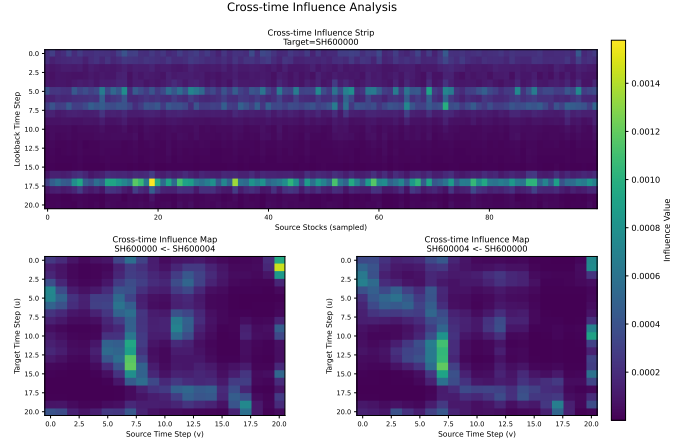


Fig. 3. Cross-time propagation heatmap at the last timestamp. The x-axis shows 100 sampled stocks, and the y-axis shows the 21-day lookback window on March 20, 2020.

drop, from 0.0575 to 0.0493, supporting the importance of explicit cross-sectional interaction.

- **w/o Auxiliary Tasks & LDL & Cross-Stock Modeling:** This temporal-only variant performs worst overall, indicating that the gains do not come from model scale alone but from the combination of cross-stock modeling and liquidity-aware multi-task learning.

To verify that the auxiliary objectives provide informative supervision rather than acting as mere regularizers, we also evaluate the auxiliary heads directly on the held-out split. The volume-shock task achieves IC/ICIR of 0.3435/2.95, while the volatility task achieves IC/ICIR of 0.5157/4.36. These results indicate that LDL learns auxiliary liquidity and risk signals that are predictive in their own right, which helps explain the gains from cross-task transfer to the return objective and, ultimately, to APO.

C. Backtest Evaluation (RQ3)

To evaluate deployment value, we conduct backtests on CSI300 over 2017–2020. The strategy uses return forecasts for stock selection and volume/volatility forecasts for liquidity-aware position sizing.

1) Experiment Setting: We form daily long-short portfolios by sorting stocks into return-based deciles, going long the top decile and short the bottom decile, and assigning weights with APO. Table III compares APO against equal-weighted, inverse-volatility risk-parity, and rank-based linear-weighting baselines; the exact baseline formulas are given in

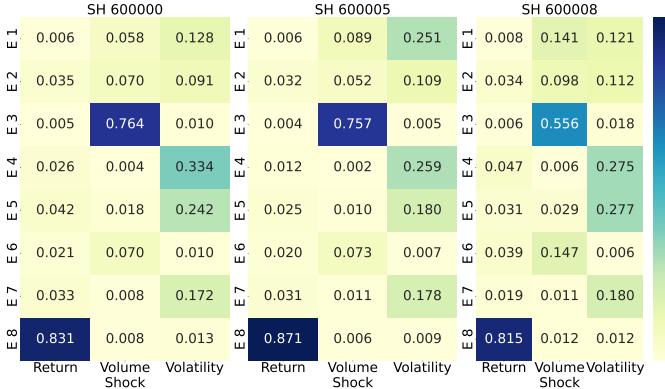


Fig. 4. MoE routing weights across tasks on March 20, 2020. The x-axis shows the three tasks, and the y-axis shows the routing weights.

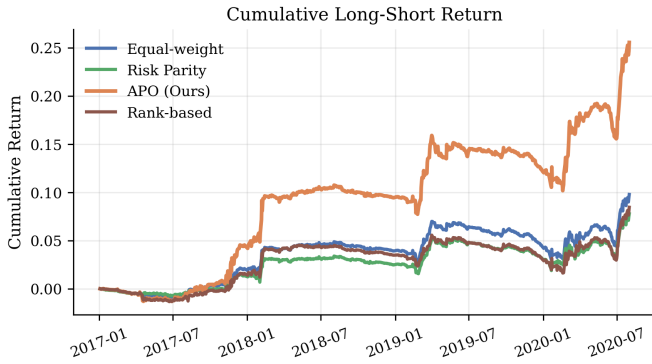


Fig. 5. Cumulative long-short returns of APO and three portfolio-construction baselines.

Appendix A-A. Each side is allocated CNY 1×10^9 , transaction cost is 10 basis points per trade, and each stock is capped at 2% of daily turnover.

2) **Experiment Performance:** Table III shows that APO (with $r = 0.1$) delivers the strongest profitability under the tested trading constraints. Relative to equal weighting, annualized return rises from 3.99% to 10.01%, OSR from 32.13% to 38.00%, and Sharpe ratio from 1.22 to 1.86; Sortino and Calmar likewise improve to 2.00 and 2.03. APO also exceeds the rank-based baseline, lifting annualized return from 3.38% to 10.01%, OSR from 30.05% to 38.00%, and Sharpe ratio from 1.18 to 1.86. These gains come with higher drawdown and turnover, indicating a deliberate return-risk trade-off rather than mechanically lower-risk allocations. The underlying model-induced long-short spread also remains significant after standard risk adjustment: over the same 2017–2020 window, a Fama-French five-factor regression still yields a daily alpha of 0.35% ($p < 0.01$). This suggests that APO improves how the signal is executed, rather than merely amplifying conventional style exposures. Figure 5 shows that APO remains above the rank-based portfolio for most of the sample, with a particularly visible separation during the February–March 2020 stress episode.

TABLE III
PERFORMANCE COMPARISON: APO VS. TRADITIONAL PORTFOLIO CONSTRUCTION BASELINES

Metric	Equal-weighted	Risk Parity	Rank-based	APO (Ours, $r = 0.1$)
OSR $\uparrow$	32.13%	29.27%	30.05%	38.00%
AR $\uparrow$	3.99%	3.19%	3.38%	10.01%
Downside Vol $\downarrow$	0.031	0.027	0.0278	0.050
MD $\downarrow$	3.73%	3.15%	3.28%	4.94%
Turnover $\downarrow$	20.73%	19.13%	19.42%	24.98%
ShR $\uparrow$	1.22	1.16	1.18	1.86
P/L $\uparrow$	1.59	1.69	1.67	1.73
SoR $\uparrow$	1.29	1.20	1.22	2.00
CR $\uparrow$	1.07	1.01	1.03	2.03

3) **Parameter Analysis:** We evaluate the sensitivity of APO to $r \in [0.1, 0.9]$. Table IV reports the resulting return, risk, and execution metrics.

The best return and risk-adjusted performance occur at $r = 0.1$. As r increases, APO places more weight on volatility control, reducing turnover and drawdown but also weakening profitability, which indicates that a mild liquidity preference is most effective in this market.

V. CONCLUSIONS AND FUTURE WORK

We presented LiMT, a hierarchical multi-task framework for stock forecasting and liquidity-aware portfolio construction. LiMT combines alternating cross-stock/temporal attention, cross-task expert routing, and a lightweight APO rule. Across CSI300 and CSI500, it achieves the best results among the compared baselines; in realistic CSI300 backtests, APO improves annualized return from 3.99% to 10.01% and Sharpe ratio from 1.22 to 1.86 over equal weighting. These results suggest that jointly modeling return, volume, and volatility can improve both predictive quality and executable portfolio construction.

Future work includes extending the framework to intra-day microstructure data, incorporating richer alternative data sources, and modeling cross-market spillovers.

APPENDIX A EVALUATION AND IMPLEMENTATION

Implementation. We implement LiMT in PyTorch on top of Qlib [28] and use Qlib implementations for all baselines. For baseline tuning, the number of layers and hidden size are searched over $\{1, 2, 3\}$ and $\{128, 256, 512\}$, and the learning rate over $\{10^{-i}\}_{i \in \{3, 4, 5, 6\}}$. For LiMT, we tune the hidden size and learning rate in the same ranges, and use $D = 256$, $lr = 10^{-5}$, $N_1 = 4$, and $N_2 = 2$ across datasets. The task weights λ_r , λ_v , and λ_σ are selected from $\{0.63, 0.65, 0.68, 0.7\}$, $\{0.3, 0.33, 0.35\}$, and $\{0.03, 0.05, 0.08\}$ according to validation IC.

Each model is trained for at most 100 epochs with early stopping on a server with an Intel Xeon Platinum 8358 CPU, 2TB memory, and one RTX3090 GPU (24GB memory). Every experiment is repeated three times with different random initializations, and the average performance is reported. For APO backtesting, we allocate CNY 1×10^9 to each of the long and short sides, set transaction cost to 10 basis points

TABLE IV
SENSITIVITY ANALYSIS OF WEIGHTING PARAMETER r .

r	OSR $\uparrow$	AR $\uparrow$	ShR $\uparrow$	MD $\downarrow$	P/L Ratio $\uparrow$	SoR $\uparrow$	CR $\uparrow$	Downside Vol $\downarrow$	Turnover Ratio $\downarrow$
0.1	0.3800	0.1001	1.8607	0.0494	1.7309	2.0030	2.0260	0.0500	0.2498
0.2	0.3774	0.0933	1.8316	0.0470	1.7100	1.9895	1.9864	0.0469	0.2479
0.3	0.3737	0.0848	1.7903	0.0438	1.6672	1.9515	1.9367	0.0435	0.2453
0.4	0.3684	0.0765	1.7649	0.0408	1.6510	1.9410	1.8738	0.0394	0.2416
0.5	0.3611	0.0685	1.7494	0.0385	1.6397	1.9501	1.7777	0.0351	0.2366
0.6	0.3509	0.0590	1.6854	0.0373	1.6560	1.8827	1.5807	0.0314	0.2299
0.7	0.3367	0.0480	1.5488	0.0355	1.6279	1.7004	1.3542	0.0283	0.2207
0.8	0.3171	0.0372	1.3691	0.0320	1.6192	1.4652	1.1621	0.0254	0.2086
0.9	0.2913	0.0262	1.1035	0.0282	1.6274	1.1405	0.9297	0.0230	0.1937

per trade, and cap the traded value of each stock at 2% of its daily turnover.

A. Backtest Portfolio Baselines

To make the portfolio-construction comparison reproducible, we define the three non-APO weighting rules used in Figure 5. Let $\mathcal{G}_t$ denote the candidate stock set on trading day t (corresponding to the long or short decile), let $C_{i,t}$ be the close price of stock i , and let $s_{i,t}$ be its predicted return score.

Equal-weighted:

$$w_{i,t}^{\text{EW}} = \frac{1}{|\mathcal{G}_t|}, \quad i \in \mathcal{G}_t. \quad (28)$$

Risk parity (inverse-volatility): we first compute one-day returns and 20-day historical volatility,

$$r_{i,t}^{(1d)} = \frac{C_{i,t}}{C_{i,t-1}} - 1, \quad \hat{\sigma}_{i,t} = \text{Std}\left(r_{i,t-k}^{(1d)}\right)_{k=0}^{19}, \quad (29)$$

then allocate weights by normalized inverse volatility:

$$w_{i,t}^{\text{RP}} = \frac{\hat{\sigma}_{i,t}^{-1}}{\sum_{j \in \mathcal{G}_t} \hat{\sigma}_{j,t}^{-1}}, \quad i \in \mathcal{G}_t. \quad (30)$$

Rank-based (linear normalization): let $\rho_{i,t} = \text{Rank}_{\mathcal{G}_t}(s_{i,t})$. We reverse the ranks for the short group and keep them unchanged for the long group:

$$\tilde{\rho}_{i,t} = \begin{cases} |\mathcal{G}_t| - \rho_{i,t} + 1, & \mathcal{G}_t = \mathcal{G}_t^{\text{short}}, \\ \rho_{i,t}, & \mathcal{G}_t = \mathcal{G}_t^{\text{long}}, \end{cases} \quad (31)$$

followed by linear normalization:

$$w_{i,t}^{\text{RK}} = \frac{\tilde{\rho}_{i,t} - \min_{j \in \mathcal{G}_t} \tilde{\rho}_{j,t}}{\sum_{j \in \mathcal{G}_t} (\tilde{\rho}_{j,t} - \min_{k \in \mathcal{G}_t} \tilde{\rho}_{k,t})}, \quad i \in \mathcal{G}_t. \quad (32)$$

APPENDIX B METRIC DEFINITIONS

For predictive evaluation, IC measures the daily correlation between predicted scores and realized returns, and ICIR normalizes IC by its temporal standard deviation. For benchmark-relative portfolio evaluation, $\text{AR}_{\text{excess}}$ and $\text{IR}_{\text{excess}}$ measure

excess profitability and its stability against the corresponding CSI benchmark.

For APO backtests, we additionally report annualized return, Sharpe ratio, Sortino ratio, Calmar ratio, maximum drawdown, downside volatility, turnover, order success rate, and profit-to-loss ratio to summarize profitability, downside risk, and execution quality.

APPENDIX C COMPUTATIONAL COMPLEXITY ANALYSIS

Let S denote the number of stocks, T the lookback length, d the hidden dimension, d_k the attention-head dimension, and n the number of experts. Cross-stock attention costs $O(TS^2d_k)$, temporal attention costs $O(ST^2d_k)$, and expert routing/aggregation costs $O(Sdn)$. Hence the total time complexity is dominated by

$$O(S \max(T^2, S)d_k + Sdn), \quad (33)$$

with space complexity

$$O(TS^2 + ST^2 + STd). \quad (34)$$

This is substantially smaller than direct pairwise cross-time interaction, which would scale as $O(S^2T^2d_k)$.

REFERENCES

- [1] F. Zhu, Z. Cai, and L. Peng, "Predictive regressions for macroeconomic data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 577–594, Mar. 2014.
- [2] M. F. H. Khan and M. M. Billah, "Macroeconomic factors and Stock exchange return: A Statistical Analysis," May 2023.
- [3] H. Wang, S. Li, T. Wang, and J. Zheng, "Hierarchical adaptive temporal-relational modeling for stock trend prediction," in *IJCAI*, 2021, pp. 3691–3698.
- [4] J. R. Quinlan, "Induction of decision trees," *ML*, pp. 81–106, 1986.
- [5] —, *C4.5: programs for machine learning*. Elsevier, 2014.
- [6] Z.-H. Zhou, *Ensemble methods: foundations and algorithms*. Chapman & Hall/CRC, 2012.
- [7] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, 2017.

- [9] C. Zhang, Y. Li, X. Chen, Y. Jin, P. Tang, and J. Li, "Doubleensemble: A new ensemble method based on sample reweighting and feature selection for financial data analysis," in *2020 IEEE international conference on data mining (ICDM)*. IEEE, 2020, pp. 781–790.
- [10] Y. Du, J. Wang, W. Feng, S. Pan, T. Qin, R. Xu, and C. Wang, "Adarnn: Adaptive learning and forecasting of time series," in *Proceedings of the 30th ACM international conference on information & knowledge management*, 2021, pp. 402–411.
- [11] H. Wang, T. Wang, S. Li, J. Zheng, S. Guan, and W. Chen, "Adaptive long-short pattern transformer for stock investment selection," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, 2022, pp. 3970–3977.
- [12] L. Zhang, C. Aggarwal, and G.-J. Qi, "Stock price prediction via discovering multi-frequency trading patterns," in *SIGKDD*, 2017, pp. 2141–2149.
- [13] S. Selvin, R. Vinayakumar, E. Gopalakrishnan, V. K. Menon, and K. Soman, "Stock price prediction using lstm, rnn and cnn-sliding window model," in *2017 international conference on advances in computing, communications and informatics (icacci)*. IEEE, 2017, pp. 1643–1647.
- [14] E. Hoseinzade and S. Haratizadeh, "Cnnpred: Cnn-based stock market prediction using a diverse set of variables," *ESA*, vol. 129, pp. 273–285, 2019.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [16] F. Feng, X. He, X. Wang, C. Luo, Y. Liu, and T.-S. Chua, "Temporal relational ranking for stock prediction," *ACM Transactions on Information Systems (TOIS)*, vol. 37, no. 2, pp. 1–30, 2019.
- [17] J. Yoo, Y. Soun, Y.-c. Park, and U. Kang, "Accurate multivariate stock movement prediction via data-axis transformer with multi-level contexts," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 2037–2045.
- [18] T. Li, Z. Liu, Y. Shen, X. Wang, H. Chen, and S. Huang, "Master: Market-guided stock transformer for stock price forecasting," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 1, 2024, pp. 162–170.
- [19] S. Xiang, D. Cheng, C. Shang, Y. Zhang, and Y. Liang, "Temporal and heterogeneous graph neural network for financial time series prediction," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 3584–3593.
- [20] S. Lin, Y. Chen, Y. Qi, C. Ma, B. Cao, Y. Zhang, X. Liu, and J. Guo, "Csp: Cross-market synergistic stock price movement forecasting with pseudo-volatility optimization," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 354–363.
- [21] R. F. Engle and G. M. Gallo, "A multiple indicators model for volatility using intra-daily data," *Journal of Econometrics*, vol. 131, no. 1-2, pp. 3–27, Mar. 2006.
- [22] J. Nettles, N. Brayer, C. Jenner, A. Ngo, C. Putnam, T. Shank, A. Todd, and P. Beling, "Forecasting intraday volume distributions," in *2015 Systems and Information Engineering Design Symposium*, Apr. 2015, pp. 97–102.
- [23] B. Á. Szűcs, "Forecasting intraday volume: Comparison of two early models," *Finance Research Letters*, vol. 21, pp. 249–258, May 2017.
- [24] C. Zhang, Y. Zhang, M. Cucuringu, and Z. Qian, "Volatility Forecasting with Machine Learning and Intraday Commonality," Rochester, NY, May 2022.
- [25] R. Goyenko, B. T. Kelly, T. J. Moskowitz, Y. Su, and C. Zhang, "Trading Volume Alpha," Rochester, NY, May 2024.
- [26] T. Ma and Y. Tan, "Multiple Stock Time Series Jointly Forecasting with Multi-Task Learning," in *2020 International Joint Conference on Neural Networks (IJCNN)*, Jul. 2020, pp. 1–8.
- [27] H. J. Park, Y. Kim, and H. Y. Kim, "Stock market forecasting using a multi-task approach integrating long short-term memory and the random forest framework," *Applied Soft Computing*, vol. 114, p. 108106, 2022.
- [28] X. Yang, W. Liu, D. Zhou, J. Bian, and T.-Y. Liu, "Qlib: An ai-oriented quantitative investment platform," *arXiv preprint arXiv:2009.11189*, 2020.
- [29] S. Hochreiter, "Long short-term memory," *Neural Computation MIT Press*, 1997.
- [30] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," in *SSST*, 2014, pp. 103–111.
- [31] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. W. Cottrell, "A dual-stage attention-based recurrent neural network for time series prediction," in *IJCAI*, 2017, p. 2627–2633.
- [32] L. Zhang, C. Aggarwal, and G.-J. Qi, "Stock price prediction via discovering multi-frequency trading patterns," in *SIGKDD*, 2017, p. 2141–2149.
- [33] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv*, 2018.
- [34] S. Ö. Arik and T. Pfister, "Tabnet: Attentive interpretable tabular learning," in *AAAI*, vol. 35, no. 8, 2021, pp. 6679–6687.
- [35] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *IJF*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [36] N. Yadati, "Localformer: Mitigating over-globalising in transformers on graphs with localised training," *Transactions on Machine Learning Research*, 2025.
- [37] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," *NeurIPS*, vol. 31, 2018.